Ollscoil
Teicneolaíochta
an Atlantaigh

Atlantic
Technological
University

Data Science

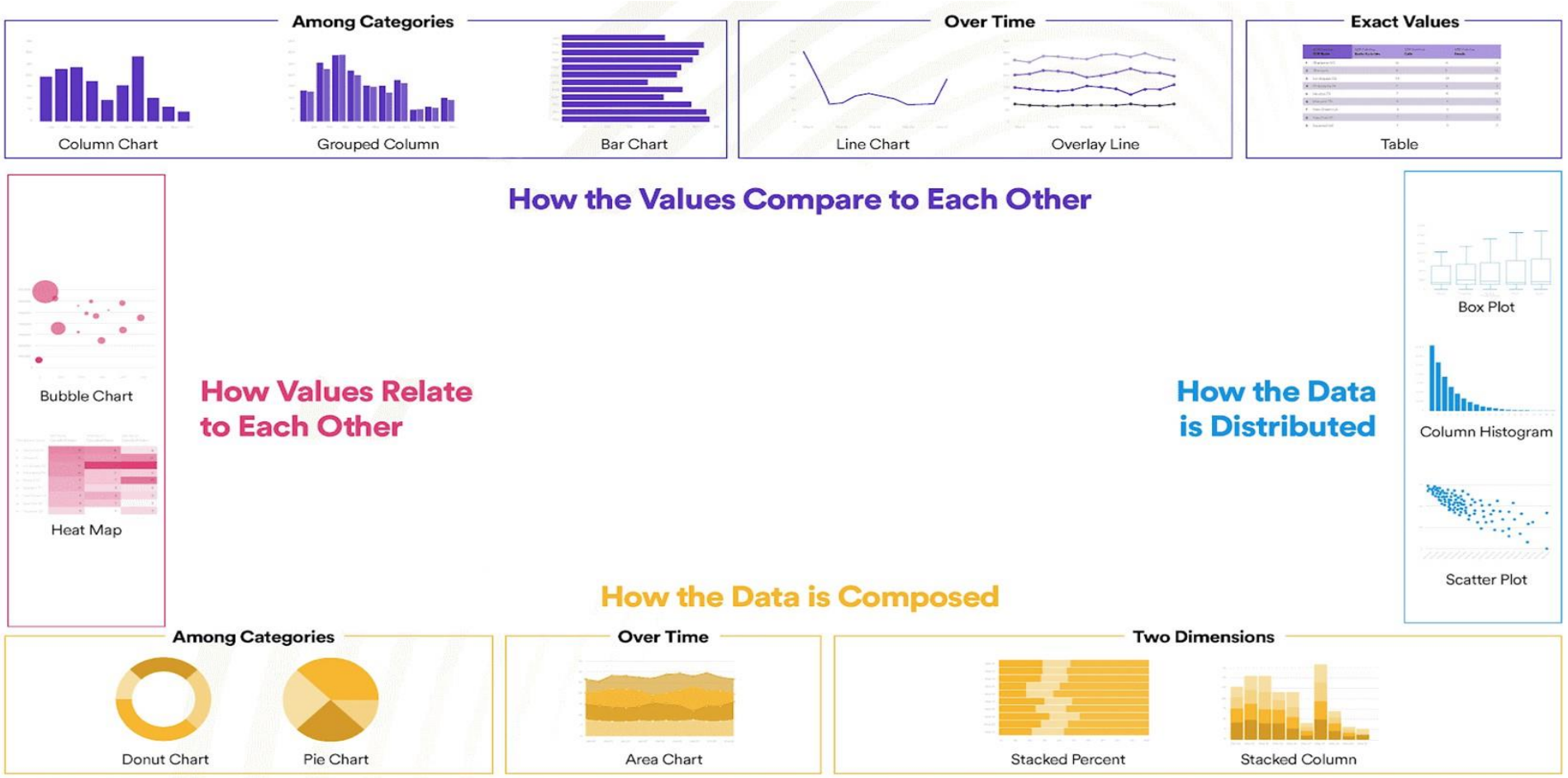
>Charts

Introduction

- Whenever we analyse data, the first thing we should do is look at it.
- R provides a wealth of functions for visualising data.

- In this lecture, we'll look at graphs that help you understand a single categorical or continuous variable, including:
  - Visualising the distribution of a variable
  - Comparing groups on an outcome variable

Chart options:



Showing comparisons

- When comparing data, we ask how different values or attributes within the data compare to each other.
- Tables help compare exact values to one another.
- Column and bar charts show comparisons across different categories.
- Line charts show trends over time.

Showing distributions

- Distribution shows how individual data points are distributed within the broader data set.
- Box plots show distribution based on a statistical summary.
- Column histograms show the frequency of an occurrence.
- Scatter plots show distribution in large data sets.

Showing composition

- Composition investigates the general features in a dataset.
- Donut and pie charts show composition when simple proportions are useful.
- Area charts put the composition of data within the context of trends over time.
- Stacked bar, percent, and column charts show an overview of the data's composition.

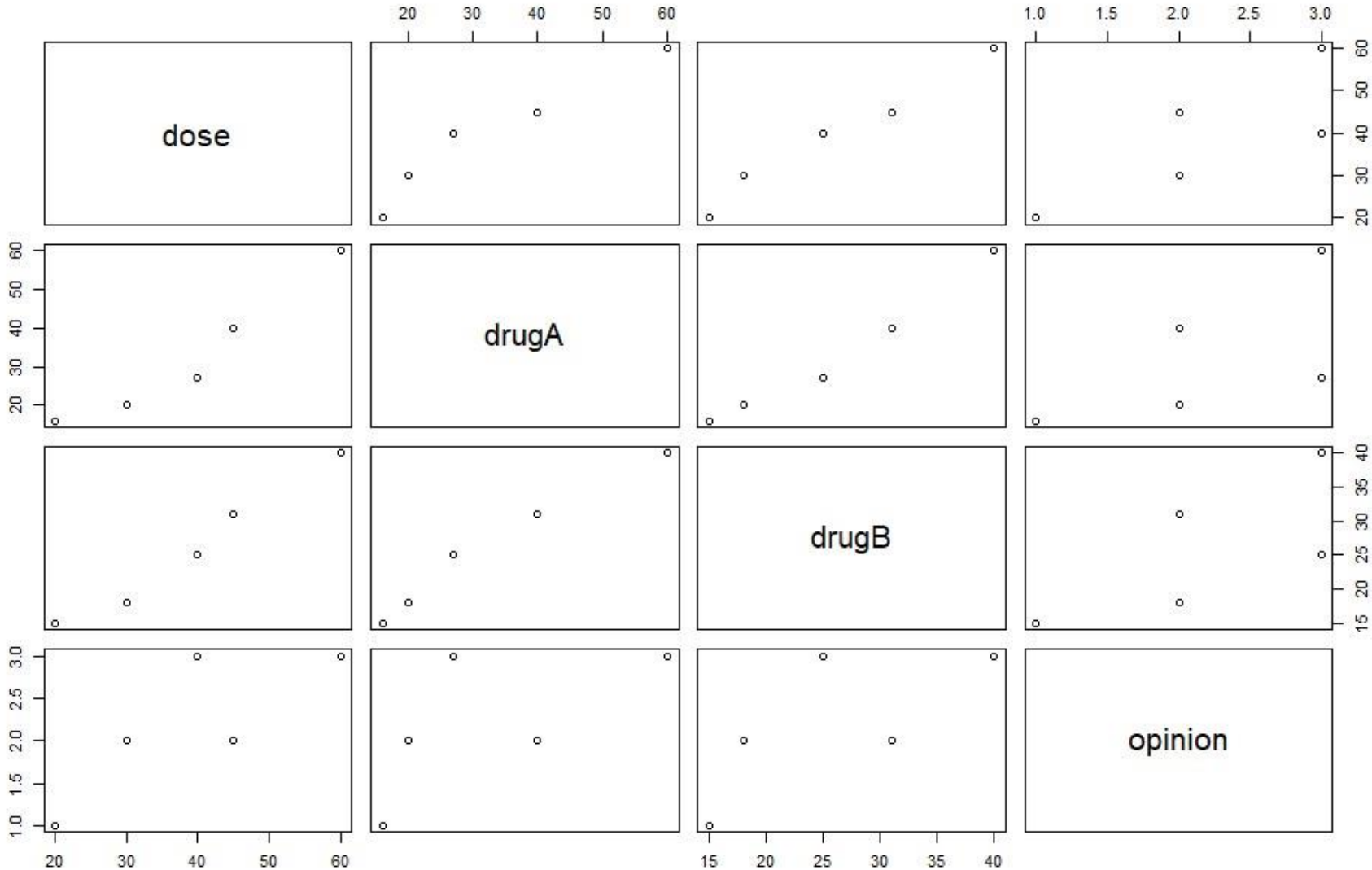
A simple example

- Try this …
- Create the following data frame. Insert your data into vectors called **dose**, **drugA** and **drugB.** Call the data frame **drugs**

Dosage (ml)	Drug A	Drug B	Opinion
20	16	15	Excellent
30	20	18	No effect
40	27	25	Some effect
45	40	31	No effect
60	60	40	Some effect

Then compare both drugs. Plot the data frame using the **plot(drugs)** function

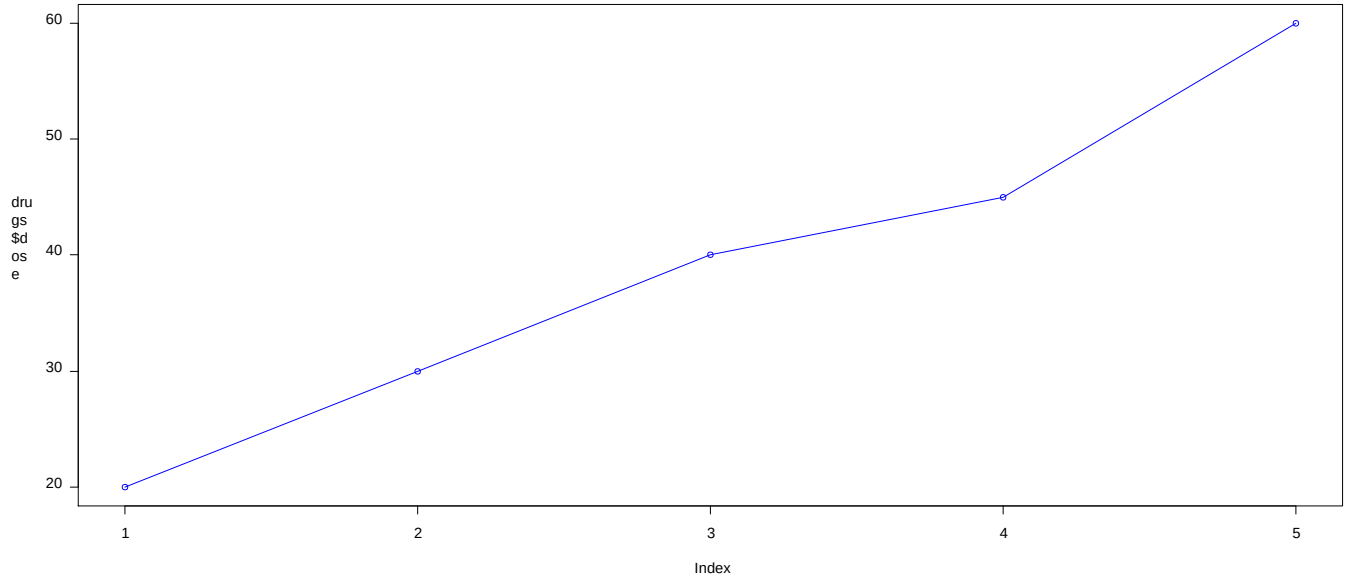
- Summary of all values plotted against each other – difficult to understand


dose
drugA
drugB
opinion
20
30
40
50
60
15
20
25
30
35
40
1.0
1.5
2.0
2.5
3.0

A simple example

- Need to analyse data individually
- Try this … Plot() **dose** using keyword **type = "o"** (for a blue dot) with a blue joining line (**col="blue")**

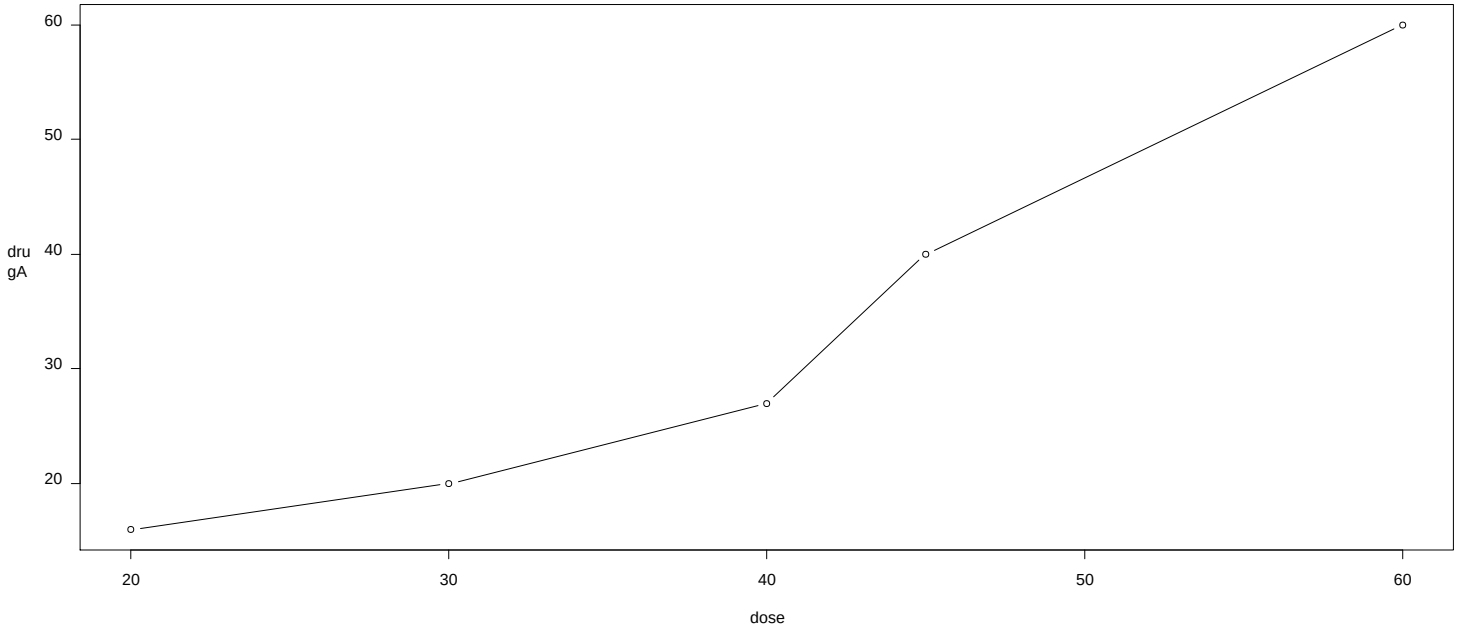
```
# Show drugs data with circles for
data points, joined with
# a blue line attach(drugs)
plot(dose, type = "o", col = "blue")
```



A simple example

- Examine info on option for **plot()** – check what "**type**" does.

- Use **type="b"** to plot both dose and drugA on the same chart.
-

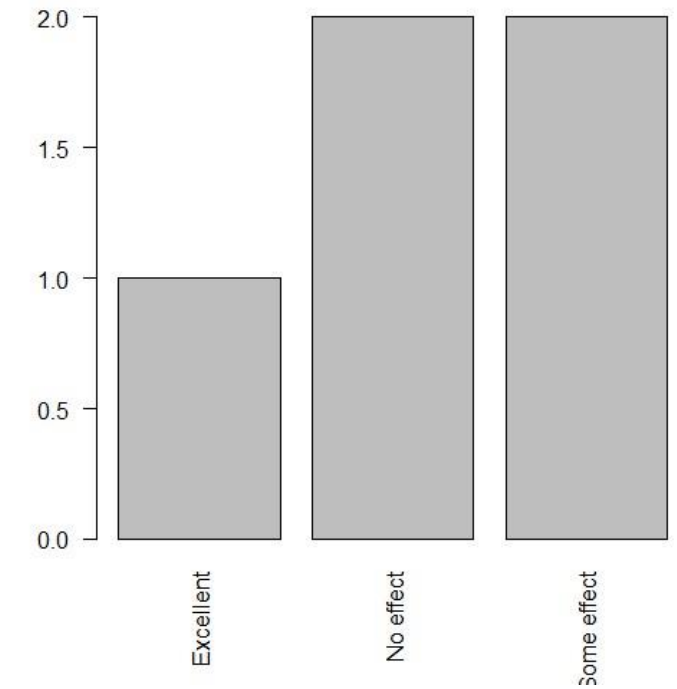



```
?plot
# option type="b" indicates that
# both points and lines should be
# plotted.
plot(dose, drugA, type = "b")
dose
```

Plotting categorical data

- Must convert categorical data for “factor” type first
- Use the as.factor() function to convert categorical variable

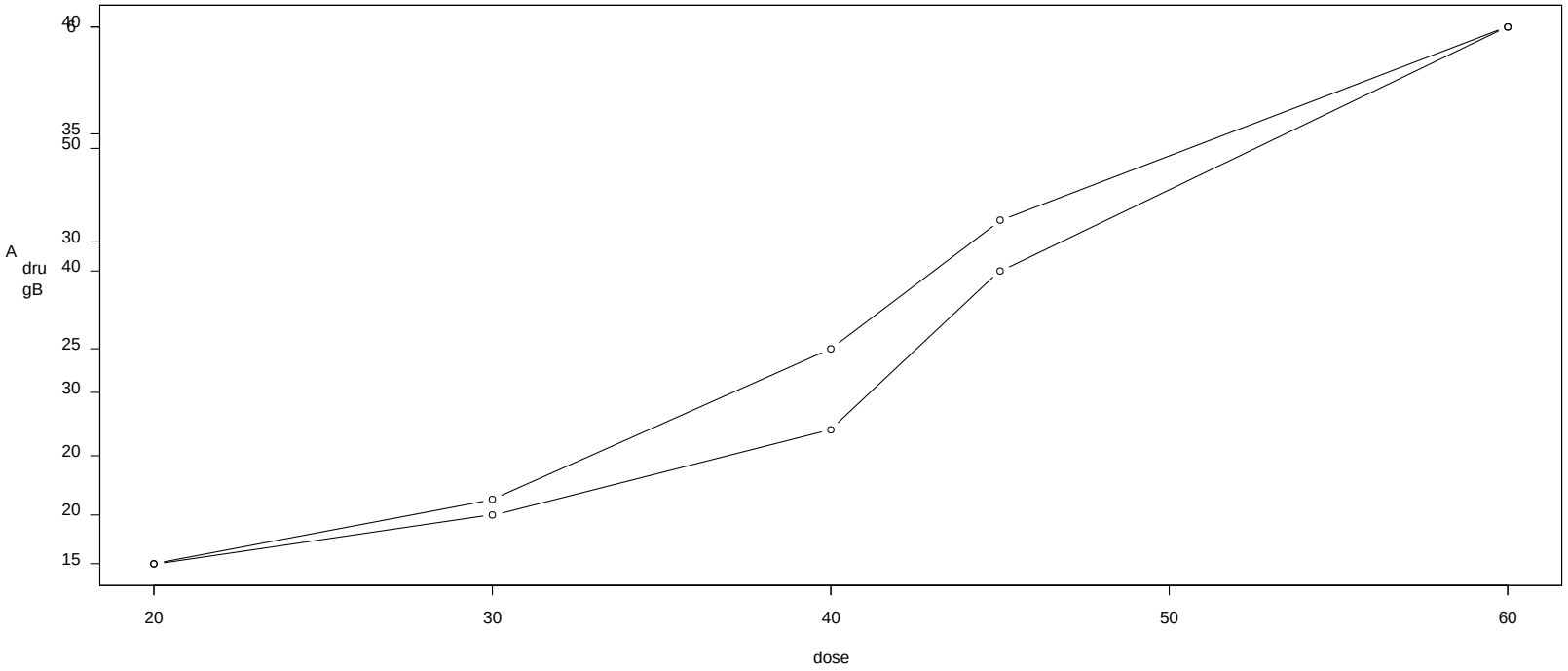
```
# If we want to plot categorical data then
# convert to factor first drugs$opinion
<- as.factor(drugs$opinion)
# las = 2 rotates x-axis
plot(drugs$opinion, las = 2)
```



Par function.

- Examine help on option for **par()** function.
- Used to customise many features on a chart such as fonts, colours, axes, labels.
- Options set with **par()** apply to all charts – whereas options set within a particular chart such as **plot()** only refer to that particular chart.
- Look at the "new" setting in **par()**.
- Set **new = true** i.e. par(new = true).
- Try this … plot drug B ie **plot(dose, drugB, type = "b")**

Example using par() function



- Both lines shown on chart.

```
?par
    # We can also plot more
than 1 # line on the chart
using par par(new = TRUE)
 plot(dose, drugB, type = "b")
```

par() function

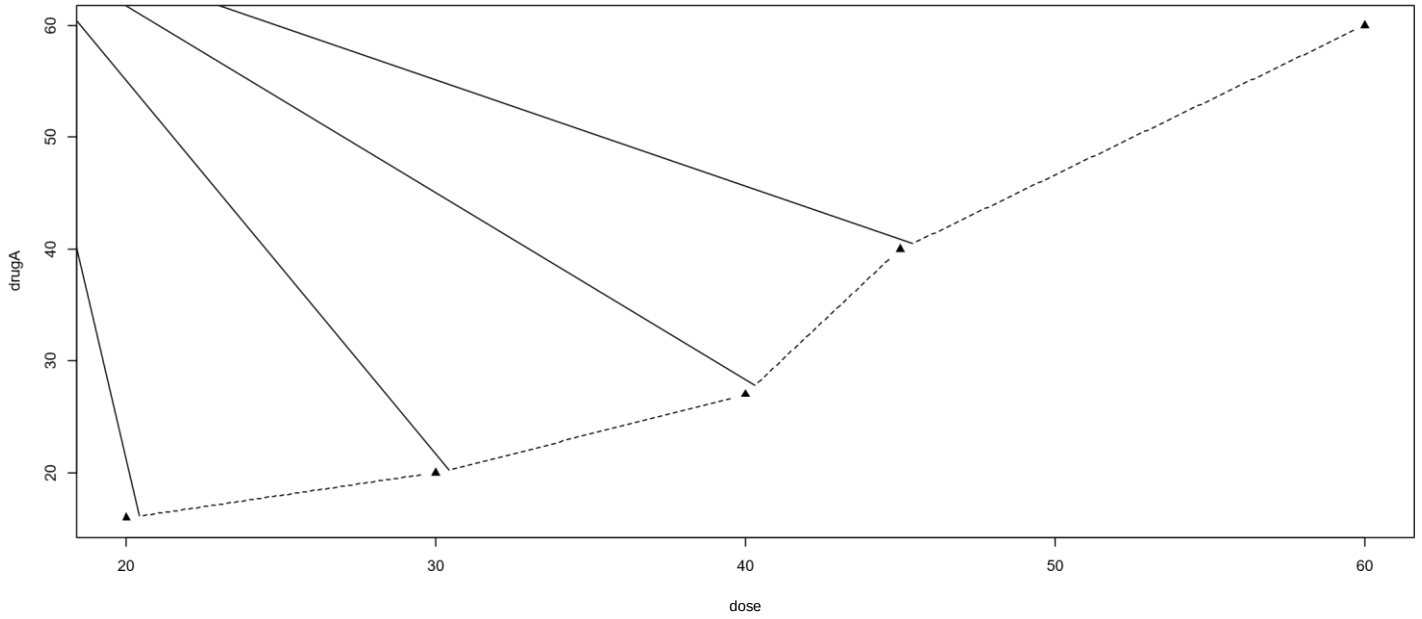
- Adding **no.readonly = TRUE** shows a list of current graphical settings that can be modified.
- Good idea to copy settings into a variable before changes are made – to reset later.
- Try this … Copy the current par settings into opar ie ….

```
opar <- par(no.readonly = TRUE)
```

- Now examine what lty does in help for par (**lty = 2** is a dashed line, **pch=17** is a solid triangle).

Configure line type and data points

- Try this
- Configure par settings with **lty=2, pch=17**, for a dashed line using a solid triangle in our next plot
- Now plot Dose on x axis and drugA on y axis, remember to also include type="b" to allow both points and lines to be plotted. Then reset par() settings to **opar** contents




```
# lty=2 dashed line
# pch=17 solid triangle
par(lty = 2, pch = 17)
plot(dose, drugA, type = "b")
par(opar)
```

par() function

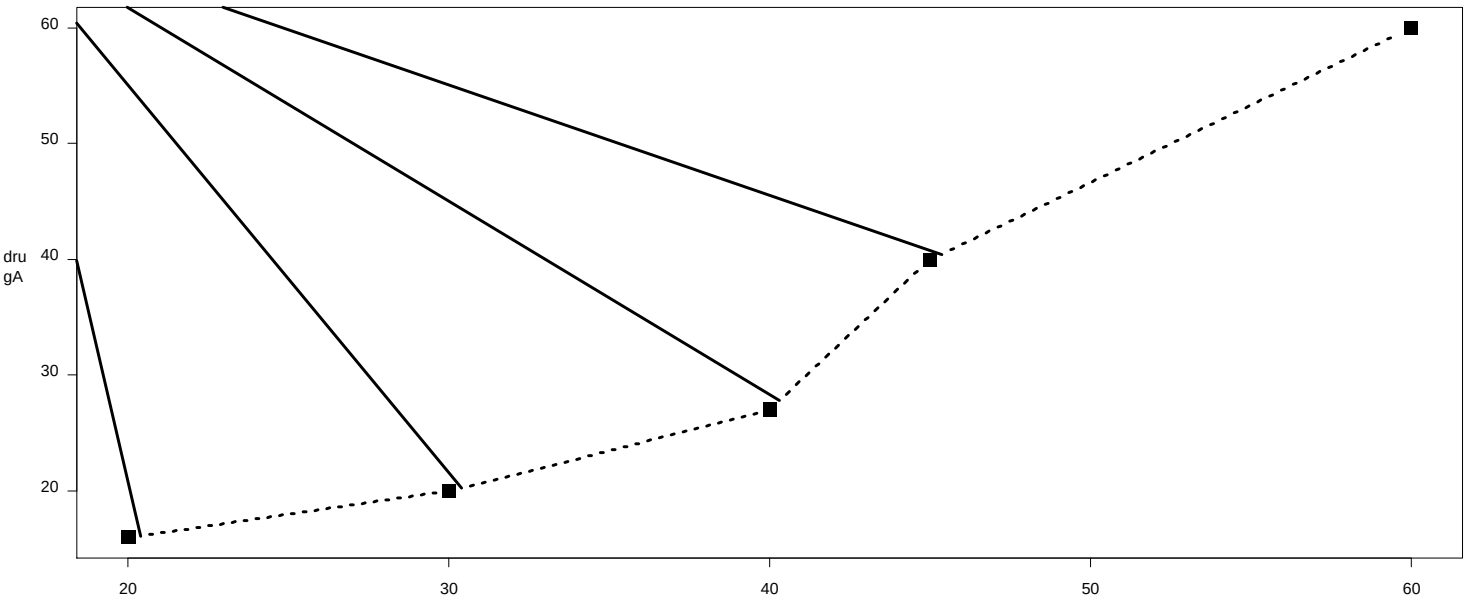
- These settings can also be included in plot() function

```
# Specify graphical parameters using
# optionname=value
plot(dose, drugA, type = "b", lty = 2, pch = 17)
```

par() function

- Try this
- Look at **plot()** help and configure a plot to show dose on x-axis, drugA on y-axis, with a dotted line which is 3 times wider than the default size. Set the connecting points to be filled squares.
- Note – you will need to examine par help

dotted line, 3 times wider than default

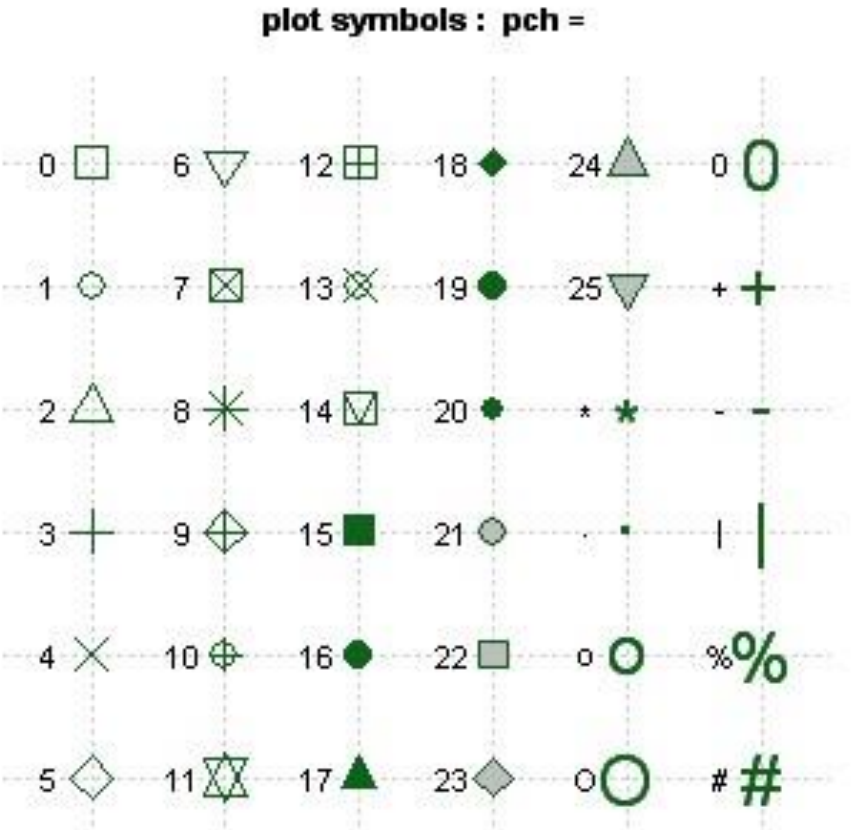


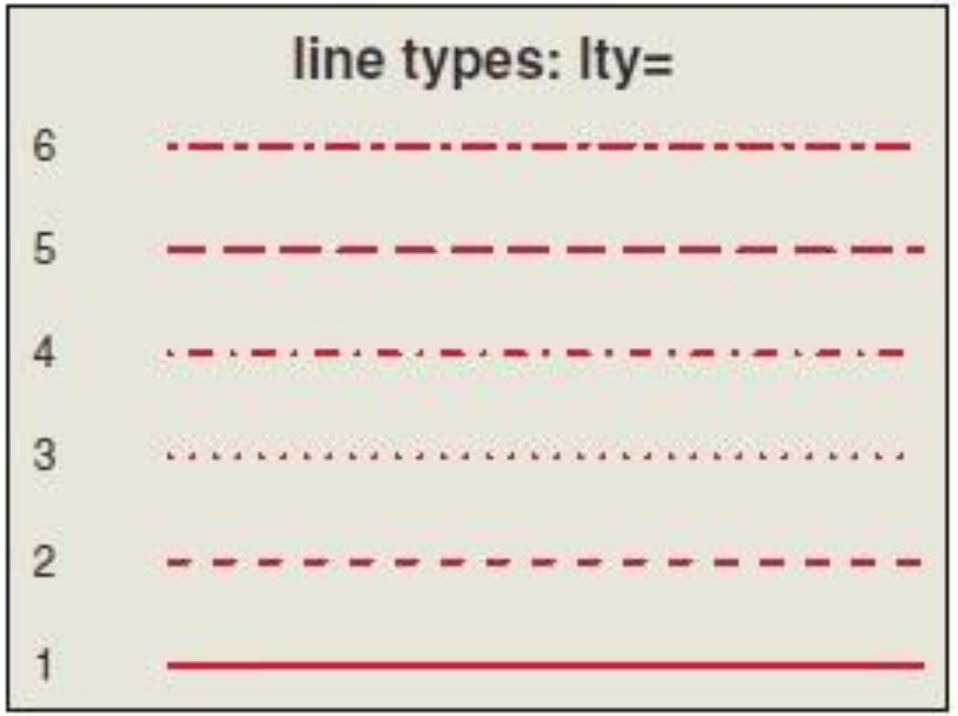
Connecting points are filled squares

```
plot(dose, drugA, type = "b", lty = 3, lwd = 3, pch = 15, cex = 2)
```

Lots of par() enhancements

Some symbols to use when plotting points. Possible values are shown here.





Adding colour and text

- Use the keyword **colors()** to show all available colours
- You can specify colours in R by index, name, hexadecimal, or RGB.
- For example col=1, col="white", and col="#FFFFFF" are equivalent.

option	**description**
col	Default plotting colour. Some functions (e.g. lines) accept a vector of values.
col.axis	colour for axis annotation
col.lab	colour for x and y labels
col.main	colour for titles
col.sub	colour for subtitles
fg	plot foreground colour (axes, boxes - also sets col= to same)

bg	plot background colour

Adding colour and text

- You can easily set font size and style, but font family is a bit more complicated.

option	**description**
font	Integer specifying font to use for text. 1=plain, 2=bold, 3=italic, 4=bold italic, 5=symbol
font.axis	font for axis annotation
font.lab	font for x and y labels
font.main	font for titles
font.sub	font for subtitles
ps	font point size (roughly 1/72 inch) text size=ps*cex
family	font family for drawing text. Standard values are "serif", "sans", "mono", "symbol".

Colours and text

- Try this

- Lets add some text and colour to the chart.
- Use the function **title()** to create a main title called "**Drug dosage**", set the main font to "**blue**", and the main font type to "**Bold Italic**".
- Hint – use **col.main** and **font.main** keywords

Atlantic Technological University

- Now let's add a red line for Drug A and specify the y-axis range directly so it will be large enough to fit the drug A data:
- Try this out…

```
# dotted line, 3 times wider than default #
Connecting points are filled squares
plot(dose, drugA, type = "b", lty = 3, lwd = 3, pch = 15, cex = 2, ylim = c(0,100))
title(main = "Drug dosage", col.main = "blue", font.main = 4)

# Graph Drug B with red dashed line and square points
lines(dose, drugB, type = "o", pch = 22, lty = 2, col = "red")
```

Atlantic Technological University

- Next let's change the axes labels to match our data and add a legend.

- So far we've been using "dose" for the x-axis and both drugs on the y-axis.
- Now we'll show the individual attributes for drugs and let **plot()** generate the index value.
- We'll also compute the y-axis values using the **max** function so any changes to our data will be automatically reflected in our graph.
- First calculate the maximum range of both vectors using the **range()** function. Store the result in **graph_range.**

```
graph_range <- range(0, drugA, drugB)
```

Atlantic Technological University

- Now graph drug A using y axis that ranges from 0 to max value in drug A or drug B vector.
- Turn off axes and annotations (axis labels) so we can specify them ourselves. • Look at help for plot to turn off the notations. **Hint** … use **axes** keyword

```
plot(drugA, type = "o", col = "blue", ylim = graph_range, axes = FALSE, ann = FALSE)
```

- Add drug b to the chart using the **lines()** function. Use a red dashed line with square points

Atlantic Technological University

```
# Graph Drug B with red dashed line and square points
lines(drugB, type = "o", pch = 22, lty = 2, col = "red")
```

- Now create a new x-axis label to show labels in millilitres

```
# Make x axis with ml labels
axis(1, at = 1:5, lab = c("20 ml", "40 ml", "60 ml", "80 ml", "100 ml"))
```

- Now create a new x-axis label to show millilitre labels

```
# Make y axis with horizontal labels that display ticks at
# every 5 marks. 5*0:graph_range[2] is equivalent to c(0, 5, 10, ..., 60).
axis(2, las = 1, at = 5 * 0:graph_range[2])
```

Atlantic Technological University

- And put a box around the chart

```
box()
```

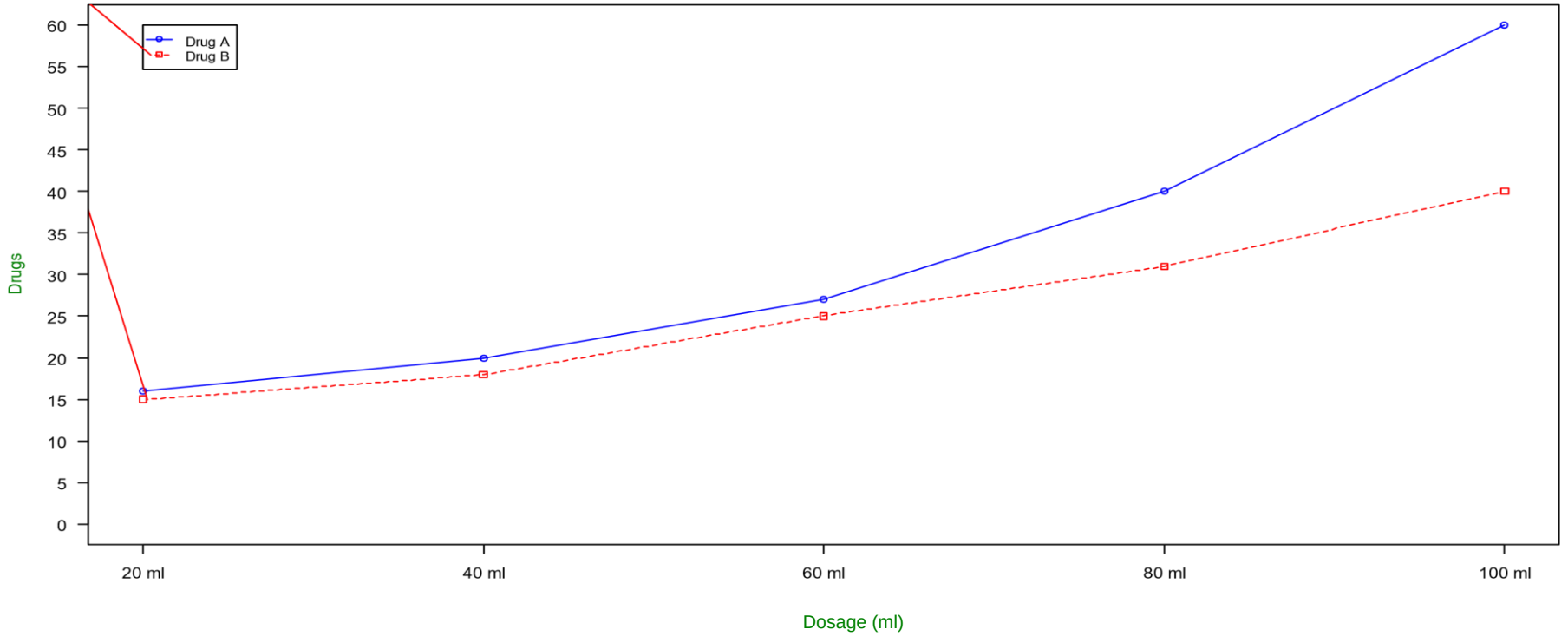
- Label the x and y axis with dark green text

```
title(xlab = "Dosage (ml)", col.lab = rgb(0, 0.5, 0))
title(ylab = "Drugs", col.lab = rgb(0, 0.5, 0))
```

- Finally create a legend with the same line colours and types to match the lines in the chart

Atlantic Technological University

- legend(1, graph_range[2], c("Drug A", "Drug B"), cex = 0.8, col = c("blue", "red"), pch = 21:22, lty = 1:2);



Atlantic Technological University

Enhancements

- Lets enhance the example
- Save the graphical parameters first
- In the example we're about to work on, we'll need to look at the par() options

```
# Save current graphical parameter settings - restore later
opar <- par(no.readonly = TRUE)
```

Atlantic Technological University